

TECH

# TYPESCRIPT

**BASICS / TYPES**    GENERICS / ADVANCED TS    TS ARCHITECTURE

## BASICS / TYPES

20 ВОПРОСОВ    7 ДНЕЙ    КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК    [typescriptlang.org/docs](https://typescriptlang.org/docs)

# 20 ВОПРОСОВ      7 ДНЕЙ

|                   |                                                                      |                       |                                                                     |
|-------------------|----------------------------------------------------------------------|-----------------------|---------------------------------------------------------------------|
| ДЕНЬ 1<br>ЛЁГКИЙ  | TYPE vs INTERFACE<br>Q1 · Q2 · Q5                                    | LITERAL TYPES         | Базовый выбор инструмента и литеральные типы — фундамент.           |
| ДЕНЬ 2<br>СРЕДНИЙ | UNION INTERSECTION<br>UNION<br>Q3 · Q4 · Q6                          | DISCRIMINATED         | Композиция типов — основа моделирования домена.                     |
| ДЕНЬ 3<br>СРЕДНИЙ | NARROWING    typeof / in / instanceof<br>TYPE GUARD<br>Q8 · Q9 · Q10 |                       | Как TS сужает union до конкретного типа в потоке кода.              |
| ДЕНЬ 4<br>СРЕДНИЙ | unknown vs any    never vs void<br>Q12 · Q14 · Q15                   |                       | Особые типы. Куда какой и почему any опасен.                        |
| ДЕНЬ 5<br>ТЯЖЁЛЫЙ | ASSERTION FUNCTION<br>Q11 · Q16                                      | EXHAUSTIVE CHECK      | Связка для гарантий: функция-утверждение и проверка-полноты switch. |
| ДЕНЬ 6<br>СРЕДНИЙ | IMPOSSIBLE STATES<br>ONAL<br>Q7 · Q13 · Q17                          | КОГДА any    OPTI     | Дизайн-практики: как не давать невозможным состояниям выразиться.   |
| ДЕНЬ 7<br>ТЯЖЁЛЫЙ | INDEXED ACCESS<br>L<br>Q18 · Q19 · Q20                               | MAPPED    CONDITIONAL | Три механизма манипуляции типами — заход в advanced.                |

РИТМ    30-60 МИН ТЕОРИЯ    15-30 МИН КОД В КОНСОЛИ    15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ    ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА ЛЁГКИЙ 3 ВОПРОСА

## TYPE vs INTERFACE LITERAL TYPES

### ЧТО УЧИМ В ЭТОТ ДЕНЬ

Базовый выбор инструмента и литеральные типы — фундамент.

### ВОПРОСЫ ДНЯ

- Q01 Чем `type` отличается от `interface`?
- Q02 Когда использовать `interface`, а когда `type`?
- Q05 Что такое `literal types`?

## Чем type отличается от interface?

### ОПИСАНИЕ

type и interface – два способа дать имя типу. interface – именованный объектный тип, расширяется через extends, поддерживает declaration merging (несколько объявлений с тем же именем сливаются). type – псевдоним любого типа: объект, union, intersection, primitive, кортеж, conditional. type не сливается. В большинстве случаев они взаимозаменяемы для объектных типов; разница появляется в углах – merging, performance компилятора, поддержка union/conditional/template-literal.

### КАК РАБОТАЕТ

- 01 interface F { ... } – именованный объектный/функциональный тип. Extends, implements – естественны.
- 02 type T = ... – псевдоним любого типа. Можно union, intersection, literal, function, tuple, conditional, mapped.
- 03 Declaration merging: два interface с одним именем объединяются. Type – нельзя, будет ошибка дубля.
- 04 interface может расширять type-объект, type может пересекать interface – между ними нет жёстких границ.
- 05 Производительность TS-компилятора: огромные intersection-типы (через type) тяжелее для tsserver, чем эквивалентный interface.
- 06 Полезное правило проекта: interface для public-API, type – для всего остального (union, конкретные shape, generic helpers).

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 interface – публичные модельные типы, расширяемые через extends.
- 02 interface – если нужен declaration merging с библиотекой (например, расширить globalThis, Express.Request).
- 03 type – union (string | number), discriminated union, tuple-типы, mapped/conditional types.
- 04 type – короткие алиасы для функций: type Handler = (e: Event) => void.
- 05 type – псевдоним для уже существующего типа: type ID = string.

### ПОДВОДНЫЕ КАМНИ

- 01 Дубль interface с тем же именем – молча сольётся, может дать неожиданное поведение в большом проекте.
- 02 Дубль type – ошибка компиляции.
- 03 interface не может выразить union напрямую (только через type-обёртку).
- 04 extends в interface – один формат; в type-через-intersection – другая семантика, особенно с конфликтующими полями.

05 Производительность: type-аналог большого intersection компилируется медленнее.

---

**КОД**

// interface – extends, merging

```
interface User { id: string }  
interface User { name: string } // declaration merging  
// итог: { id: string; name: string }  
interface Admin extends User { role: 'admin' }
```

// type – union, intersection, alias

```
type ID = string;  
type Status = 'idle' | 'loading' | 'done';  
type Admin = User & { role: 'admin' };  
type Handler = (e: Event) => void;
```

---

**МОГУТ ДОСПРОСИТЬ**

- 01 Что такое declaration merging и зачем оно?
- 02 Почему extends в interface дешевле, чем intersection через type?
- 03 Можно ли interface использовать для union?

---

**ПОЛНАЯ СТАТЬЯ**

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

## Когда использовать `interface`, а когда `type`?

### ОПИСАНИЕ

Чёткого правила в стандарте нет — оба инструмента закрывают пересекающиеся задачи. Эмпирика: `interface` — когда тип представляет «форму объекта» и есть шанс расширения (`extends`, `declaration merging`); `type` — когда тип сложнее объекта (`union`, `intersection`, `conditional`, `mapped`) или это псевдоним к существующему типу. В TS-командах часто договариваются «`interface` для `public API`, `type` для всего остального» — это даёт согласованность.

### КАК РАБОТАЕТ

- 01 `interface` проще читается в ошибках компилятора (короткое имя).
- 02 `type` быстрее «разворачивается» компилятором, что иногда удобно для отладки сложных типов.
- 03 Для публичного API библиотеки — `interface`, чтобы клиенты могли расширять через `declaration merging`.
- 04 Для `domain-моделей` внутри проекта — обычно `type` или `interface`, на вкус команды.
- 05 Для `union/discriminated union` — только `type`.
- 06 Для функциональных типов — оба работают, но `type` читается чище: `type Handler = (e: Event) => void`.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 `interface User { ... }` — модель с возможностью `extends`.
- 02 `type Status = 'idle' | 'loading'` — `union`, только `type`.
- 03 `type Coords = [number, number]` — `tuple`, только `type`.
- 04 `type Result<T> = { ok: true; value: T } | { ok: false; error: Error }` — `discriminated union`.
- 05 `interface Window { myApp: MyApp }` — расширение глобального интерфейса через `merging`.

### ПОДВОДНЫЕ КАМНИ

- 01 Смешанная конвенция в проекте быстро становится бардаком — договариваться единым правилом.
- 02 `type` для объекта тоже работает, но теряется `declaration merging` и иногда компилятор показывает развёрнутый `shape` вместо имени.
- 03 `interface` не описывает `union` — нужна обёртка через `type`.

### КОД

```
// Обе формы для объекта — почти эквивалентны
interface IUser { id: string; name: string }
```

```
type    TUser = { id: string; name: string };  
// разница появляется в edge cases (merging, perf)
```

// type обязателен

```
type Status = 'idle' | 'loading' | 'done';  
type Pair = [number, number];  
type ResultOf<T> = T extends Promise<infer U> ? U : never;
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Какое правило выбора используете в большом проекте?
- 02 Что выбрать для public API библиотеки и почему?
- 03 Когда type читается явно лучше interface?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#differences-between-type-aliases-and-interfaces>

## Что такое literal types?

### ОПИСАНИЕ

Literal types – это типы, состоящие из ОДНОГО конкретного значения: 'idle', 42, true. Они уже типы, а не значения. Полезны как «строки-теги» для discriminated union, для ограничения возможных значений переменной/аргумента, для точного описания API. Часто комбинируются с union: 'GET' | 'POST' | 'PUT' | 'DELETE'.

### КАК РАБОТАЕТ

- 01 String literal: 'idle', number literal: 42, boolean literal: true.
- 02 Literal type – подтип соответствующего primitive: 'idle' extends string.
- 03 const-объявление переменной с примитивом сужает её тип до literal: const x = 'a' → тип 'a', а не string.
- 04 Параметры функции и поля let-объявлений по умолчанию расширяются до primitive – нужно явно указать literal или использовать as const.
- 05 Литералы используются как «теги» в discriminated union: поле type: 'add' | 'remove'.
- 06 as const «замораживает» структуру в литералах: { a: 1 } as const → { readonly a: 1 }.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Перечисления допустимых значений: type Method = 'GET' | 'POST' | 'PUT' | 'DELETE'.
- 02 Discriminated union: type Action = { type: 'add'; ... } | { type: 'remove'; ... }.
- 03 Точная типизация констант: const COLORS = ['red', 'blue'] as const → readonly ['red', 'blue'].
- 04 Литералы для props компонентов: variant?: 'primary' | 'secondary'.

### ПОДВОДНЫЕ КАМНИ

- 01 let-переменная расширяется до примитива – теряется literal: let s = 'a' → string. Нужно const или явный тип.
- 02 Без as const массив-константа имеет тип string[], а не ['a', 'b'].
- 03 Литералы True/False могут вести себя неожиданно при coercion. Например, in narrowing.
- 04 Огромные union-литералы (сотни строк) замедляют компилятор.

### КОД

// Базовое

```
type Method = 'GET' | 'POST' | 'PUT' | 'DELETE';
```



```
function fetch(url: string, m: Method): void {}  
fetch('/x', 'POST');    // ok  
fetch('/x', 'POSt');    // ошибка
```

// as const – заморозка литералов

```
const COLORS = ['red', 'blue'] as const;  
type Color = typeof COLORS[number];  
// 'red' | 'blue'
```

// Discriminated union через literal

```
type Action =  
  | { type: 'add';    item: string }  
  | { type: 'remove'; id:  string };
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Чем literal type отличается от enum?
- 02 Что делает as const с массивом и объектом?
- 03 Почему let s = 'a' имеет тип string, а не 'a'?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#literal-types>

**UNION****INTERSECTION****DISCRIMINATED UNION****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Композиция типов — основа моделирования домена.

**ВОПРОСЫ ДНЯ**

Q03 Что такое union type?

Q04 Что такое intersection type?

Q06 Что такое discriminated union?

## ВОПРОС Q03

## Что такое union type?

### ОПИСАНИЕ

Union (объединение) – тип, который может быть одним из перечисленных. Пишется через `|`. Пример: `string | number`. Значение этого типа гарантированно соответствует одному из вариантов, но компилятор не знает какому именно – для работы со специфичной частью нужен `narrowing` (см. Q8).

### КАК РАБОТАЕТ

- 01 `type X = A | B` – значение может быть A или B.
- 02 До `narrowing` доступны только общие свойства/методы обоих вариантов.
- 03 Union между `primitive` и `object`: TS требует явно проверять тип перед обращением к специфичному методу.
- 04 `Narrowing` – `typeof`, `in`, `instanceof`, `type guards`, `discriminated union`.
- 05 Union полезен для опциональных значений (`T | undefined`), результата (`Result | Error`) и `enum`-подобных констант.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Опциональные значения: `const x: string | null`.
- 02 Множественные источники: `function fetch(url: string | URL)`.
- 03 `Discriminated union` для вариантов состояния.
- 04 Параметры обработчиков: `id: string | number`.

### ПОДВОДНЫЕ КАМНИ

- 01 Без `narrowing` нельзя использовать специфичные свойства.
- 02 Union с пересекающимися полями – TS сужает до общего пересечения.
- 03 `string | number` – нельзя сравнить через `===`, если нужна конкретика (нужно `typeof`).
- 04 Большие union (10+ вариантов) – обычно знак, что нужен `discriminated union`.

### КОД

// Базовое

```
function format(x: string | number): string {
  if (typeof x === 'number') return x.toFixed(2); // narrowed → number
  return x.trim();                               // narrowed → string
}
```

// Опциональный с null

```
function load(): User | null { ... }
const u = load();
if (u) u.name; // narrowed
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Чем union отличается от intersection?
- 02 Почему до narrowing доступны только общие поля?
- 03 Когда union превращать в discriminated union?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#union-types>

## ВОПРОС Q04

## Что такое intersection type?

### ОПИСАНИЕ

Intersection (пересечение) – тип, который объединяет несколько типов в один: A & B. Значение должно удовлетворять одновременно A И B – содержать все поля обоих. В отличие от union (или одно, или другое), intersection требует «и то, и то». Часто используется для композиции type-объектов и для добавления свойств к существующему типу.

### КАК РАБОТАЕТ

- 01 type X = A & B – значение должно соответствовать обоим.
- 02 Для объектных типов intersection даёт объект со всеми полями обеих сторон.
- 03 Если поля конфликтуют по типу – intersection становится never (для primitive) или сложным пересечением вложенных типов.
- 04 Intersection примитивов: string & number = never (нет значения).
- 05 interface extends – обычно эквивалент intersection, но extends быстрее и понятнее.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Композиция: type AdminUser = User & { role: 'admin' }.
- 02 Добавление полей к существующему типу без расширения interface.
- 03 Слияние props компонента с базовыми: props & HTMLAttributes<HTMLDivElement>.
- 04 Generic-хелперы: type WithId<T> = T & { id: string }.

### ПОДВОДНЫЕ КАМНИ

- 01 Конфликт полей превращает их в never – могут быть тонкие ошибки.
- 02 Intersection примитивов – never. string & number = never.
- 03 Большие intersection нагружают компилятор; extends в interface – быстрее.
- 04 Не путать с union: A | B значит «одно из», A & B – «и то и то».

### КОД

// Композиция объектов

```
type User = { id: string; name: string };
type Admin = User & { role: 'admin' };
const a: Admin = { id: '1', name: 'A', role: 'admin' };
```

// Конфликт = never

```
type X = { x: string } & { x: number };
const a: X = { x: ??? }; // x: never
```

### МОГУТ ДОСПРОСИТЬ

- 01 Что вернёт string & number?

02 Чем intersection отличается от extends?

03 Что произойдёт при конфликте полей?

---

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/objects.html#intersection-types>

## ВОПРОС Q06

## Что такое discriminated union?

### ОПИСАНИЕ

Discriminated union – паттерн union из объектов с общим тегом-полем (discriminator). Тег – обычно literal type (строка). Это позволяет TS однозначно определить, с каким вариантом мы работаем, через простую проверку поля. Канонический пример – Result-тип { ok: true; value } | { ok: false; error }.

### КАК РАБОТАЕТ

- 01 Каждый вариант – объект с уникальным значением общего поля-discriminator.
- 02 Discriminator – обычно поле type или kind с literal-значением.
- 03 Сужение через if (x.type === 'add') – TS точно знает, какой вариант.
- 04 Идеален для exhaustive check (Q16) через switch.
- 05 Гарантирует, что нельзя случайно перепутать поля одного варианта с другим (impossible state).

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Action-types в Redux/state-machines.
- 02 Result-тип { ok | error }.
- 03 Состояния запроса: { state: 'loading' } | { state: 'success'; data } | { state: 'error'; error }.
- 04 Состояния формы.
- 05 WebSocket-сообщения с разными payload'ами.

### ПОДВОДНЫЕ КАМНИ

- 01 Discriminator должен быть literal – иначе narrowing не работает.
- 02 Если забыть один из вариантов в switch – exhaustive check не сработает; нужен defaultCase: never.
- 03 Без discriminator – обычный union, narrowing сложнее (in, type guards).
- 04 Дискриминатор в массиве объектов – TS суживает один элемент, а не весь массив.

### КОД

// Базовое – Action

```
type Action =  
  | { type: 'add'; item: string }  
  | { type: 'remove'; id: string }  
  | { type: 'clear' };  
  
function reduce(a: Action) {  
  switch (a.type) {
```

```
    case 'add':    return a.item;
    case 'remove': return a.id;
    case 'clear':  return null;
  }
}
```

// Result-тип

```
type Result<T, E = Error> =
  | { ok: true;  value: T }
  | { ok: false; error: E };
```

---

МОГУТ ДОСПРОСИТЬ

- 01 Зачем нужен exhaustive check рядом с DU?
- 02 Что выбрать в качестве discriminator-поля?
- 03 Можно ли использовать DU для async-state?

---

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions>



ДЕНЬ 3 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

**NARROWING    typeof / in / instanceof    TYPE GUARD****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Как TS сужает union до конкретного типа в потоке кода.

**ВОПРОСЫ ДНЯ**

Q08 Что такое narrowing?

Q09 Как работают typeof, in, instanceof narrowing?

Q10 Что такое user-defined type guard?

## Что такое narrowing?

### ОПИСАНИЕ

Narrowing (сужение типа) – процесс, в котором TS, опираясь на проверки в коде, в конкретной ветке делает тип переменной более узким, чем объявленный. Например, `x: string | number` после `if (typeof x === 'string')` внутри ветки имеет тип `string`. TS поддерживает целый набор «type guard»-конструкций, которые он понимает: `typeof`, `in`, `instanceof`, `equality (=== / ==)`, `truthy`, `user-defined guards`.

### КАК РАБОТАЕТ

- 01 TS строит control-flow analysis по коду и сужает тип в каждой ветке `if/switch/||`.
- 02 `typeof x === 'string'` – сужает к `string`. Остальные варианты уходят в `else`.
- 03 `in` – сужает по наличию ключа: `if ('foo' in x)`.
- 04 `instanceof` – для классов: `if (x instanceof Error)`.
- 05 `Equality` – сужение по сравнению: `if (x === 'add')`.
- 06 `Truthy/falsy` – сужение по истинности: `if (x)`.
- 07 `User-defined type guard` – функция, возвращающая `x is T`.
- 08 `Assertion function` – функция, утверждающая (см. Q11).

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Везде, где есть union-тип и нужно работать со специфичной частью.
- 02 `Discriminated union` – почти всегда narrowing через `equality`.
- 03 Опциональные поля – narrowing через `truthy`.
- 04 `Class hierarchy` – narrowing через `instanceof`.

### ПОДВОДНЫЕ КАМНИ

- 01 Narrowing «забывается» при асинхронном вызове: после `await` TS пере-расширяет переменную, потому что состояние могло измениться.
- 02 Narrowing внутри `callback` – TS не пробрасывает narrowing из внешней функции в `callback`.
- 03 Сложные guards (через `.some()`, `.find()`) TS не понимает – нужно явно указывать тип через `user-defined guard`.
- 04 Narrowing объекта по полю не работает, если поле не `readonly`: TS считает, что значение могло измениться.

### КОД

```
// typeof
```

```
function f(x: string | number) {
  if (typeof x === 'string') return x.trim();
}
```

```
    return x.toFixed(2);  
  }
```

// in

```
function speak(a: { bark: () => void } | { meow: () => void }) {  
  if ('bark' in a) a.bark(); else a.meow();  
}
```

// Equality (discriminated union)

```
switch (action.type) {  
  case 'add': /* action: { type: 'add'; ... } */  
  case 'remove': /* action: { type: 'remove'; ... } */  
}
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Какие виды narrowing понимает TS из коробки?
- 02 Почему narrowing «забывается» после await?
- 03 Что делать, если TS не сужает после своей проверки?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html>

## Как работают `typeof`, `in`, `instanceof` narrowing?

### ОПИСАНИЕ

Это три встроенных способа сужения, которые TS понимает без дополнительных аннотаций. `typeof` работает с примитивами (`string`, `number`, `boolean`, `bigint`, `symbol`, `undefined`, `function`, `object`). `in` — проверяет наличие ключа в объекте, сужая к варианту с этим ключом. `instanceof` — для проверки принадлежности к классу через цепочку прототипов.

### КАК РАБОТАЕТ

- 01 `typeof` возвращает одну из строк: `'string' | 'number' | 'bigint' | 'boolean' | 'symbol' | 'undefined' | 'function' | 'object'`. Нет `'array'` и `'null'` — оба `object`.
- 02 `in` проверяет ключ в объекте (включая прототипы). Удобно для `union` без `discriminator`.
- 03 `instanceof` проверяет цепочку прототипов: `x instanceof C` — `true`, если `C.prototype` есть в `[[Prototype]] x`.
- 04 TS использует эти проверки для `type narrowing` внутри `if/else`.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 `typeof` — для примитивных `union` (`string | number`).
- 02 `in` — для разделения объектных вариантов по полю-маркеру без явного `tag`.
- 03 `instanceof` — `class-based` код: разбираем `Error` / `ValidationError` / `NetworkError`.
- 04 Часто комбинируют: `typeof` + `instanceof` для смеси примитива и объекта.

### ПОДВОДНЫЕ КАМНИ

- 01 `typeof null === 'object'` — `null` не отделится от объектного варианта без отдельной проверки.
- 02 `typeof [] === 'object'` — массив тоже `object`. Для проверки массива — `Array.isArray`.
- 03 `in` проверяет наследованные ключи — может «найти» поле, которого формально нет в объекте.
- 04 `instanceof` через несколько `realm` (`iframes`) ломается, потому что `class` — другой объект в другом окне.

### КОД

// `typeof` — primitives

```
function show(x: string | number | undefined) {
  if (typeof x === 'undefined') return '?';
  if (typeof x === 'number')    return x.toFixed();
  return x.toUpperCase();
}
```

// `in` — без `discriminator`

```
type Cat = { meow(): void };
type Dog = { bark(): void };
function speak(a: Cat | Dog) {
  if ('bark' in a) a.bark(); else a.meow();
}
```

// instanceof – классы

```
function handle(e: Error) {
  if (e instanceof ValidationError) {
    /* e: ValidationError */
  }
}
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Почему `typeof null === 'object'`?
- 02 Как отличить массив от объекта в `narrowing`?
- 03 Что не так с `instanceof` в `multi-realm` среде?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#typeof-type-guards>

## Что такое user-defined type guard?

### ОПИСАНИЕ

User-defined type guard – функция, у которой возвращаемый тип объявлен в виде `arg is T`. TS трактует true-результат такой функции как доказательство того, что `arg` имеет тип `T`. Это способ научить компилятор сужать там, где встроенных проверок недостаточно – например, при проверке формы объекта или вызовом сложной функции.

### КАК РАБОТАЕТ

- 01 `function isUser(x: unknown): x is User { return ... } – сигнатура.`
- 02 Внутри тела можно делать любые проверки: `typeof`, поле, `instanceof`.
- 03 TS доверяет вашему коду – если функция вернула `true`, сужение применяется. Поэтому проверка должна быть честной.
- 04 Часто используется в массивах: `arr.filter(isUser) → User[]`.
- 05 Можно объявить как метод класса: `pet.isCat(): this is Cat`.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Сужение `arr.filter(isUser)` – превращение в типобезопасный массив.
- 02 Проверка JSON-объекта на соответствие модели после `fetch`.
- 03 Type-narrowing после внешней библиотеки (например, `isAxiosError`).
- 04 Гибкие проверки внутри `union` без явного `discriminator`.

### ПОДВОДНЫЕ КАМНИ

- 01 Если внутри функции проверка неполная – TS всё равно будет считать тип сужённым. Это лазейка для багов.
- 02 Лучше комбинировать с runtime-валидацией (`Zod`, `Valibot`).
- 03 `filter` с user-defined guard – самый удобный способ очистки массивов от `null/undefined`.
- 04 Метод-guard (`this is X`) полезен в OOP-стиле, но усложняет наследование.

### КОД

// User-guard

```
function isUser(x: unknown): x is User {
  return typeof x === 'object'
    && x !== null
    && 'id' in x
    && typeof (x as any).id === 'string';
}
```

// filter – типобезопасный

```
const arr: (User | null)[] = [...];
```

```
const onlyUsers = arr.filter((x): x is User => x !== null);
```

// Метод-guard

```
class Pet {  
  isCat(): this is Cat { return this.kind === 'cat'; }  
}
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Что произойдёт, если type guard «соврёт»?
- 02 Чем хорош filter((x): x is User => Boolean(x))?
- 03 Как комбинировать type guard с Zod?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#using-type-predicates>

## unknown vs any      never vs void

### ЧТО УЧИМ В ЭТОТ ДЕНЬ

Особые типы. Куда какой и почему any опасен.

### ВОПРОСЫ ДНЯ

Q12 Чем unknown отличается от any?

Q14 Чем never отличается от void?

Q15 Где используется never?



## ВОПРОС Q12

## Чем unknown отличается от any?

### ОПИСАНИЕ

`any` – отключение проверки типов: со значением `any` можно делать что угодно, и TS не возражает. Это «дыра» в системе типов. `unknown` – безопасный аналог: значение есть, но до явной проверки/каста с ним нельзя ничего делать. `unknown` – рекомендованная замена `any` для входных точек, где тип неизвестен (`fetch-результат`, `JSON.parse`, `catch-error`).

### КАК РАБОТАЕТ

- 01 `any` – сверху и снизу типовой иерархии: совместим со всем, и всё совместимо с ним. Никаких проверок.
- 02 `unknown` – `top type`: всё сужается до `unknown`, но из `unknown` ничего не вытекает без проверки.
- 03 С `unknown` нельзя сразу обращаться к свойствам, вызывать как функцию, выполнять арифметику.
- 04 Разрешённые операции: `typeof`, `instanceof`, `type guards`, `as-кастинг`.
- 05 TS 4.4+ – `catch-error` по умолчанию `unknown`.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 `fetch+json`: `const data: unknown = await res.json();` затем проверка через `guard` или `Zod`.
- 02 `JSON.parse` – раньше возвращал `any`, сейчас часто оборачивают в `unknown`.
- 03 `catch (e)` – стандартный TS даёт `unknown`, нужно проверить `instanceof Error`.
- 04 `Generic`-хелперы, где тип входа неизвестен заранее.

### ПОДВОДНЫЕ КАМНИ

- 01 `any` проникает по всему коду через `assignment` – разрушает проверки во многих местах. Включайте `noImplicitAny` / `strict`.
- 02 `unknown` без `guard` невозможно использовать – иногда разработчики из лени делают `as any`.
- 03 `as unknown as T` – обходной путь для двух несовместимых типов; используйте только когда уверены.
- 04 В `catch (e)` для `legacy`-кода нужно `e!`: `any` или ручная проверка `instanceof Error`.

### КОД

// `unknown` – безопасное приведение

```
async function load(): Promise<User> {
  const data: unknown = await (await fetch('/u')).json();
  if (isUser(data)) return data;
  throw new Error('bad payload');
```

```
}
```

// any – опасно

```
function f(x: any) {  
  x.b.c.d.e; // компилируется, упадёт в рантайме  
}
```

// catch – unknown по умолчанию

```
try { ... } catch (e) {  
  if (e instanceof Error) console.log(e.message);  
}
```

---

МОГУТ ДОСПРОСИТЬ

- 01 Почему any опасен в большом проекте?
- 02 Как проверять unknown без runtime-валидации?
- 03 Что делает useUnknownInCatchVariables?

---

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#the-any-type>

## ВОПРОС Q14

## Чем `never` отличается от `void`?

### ОПИСАНИЕ

`void` – тип для функции, которая ничего не возвращает (`return` без значения или `return` отсутствует). `never` – тип для функции, которая **ВООБЩЕ** не возвращает – потому что выбрасывает исключение, бесконечный цикл или останавливает процесс. С точки зрения системы типов: `void` допускает `undefined` как значение, `never` – нет значений вообще, это «пустой» тип.

### КАК РАБОТАЕТ

- 01 `void: () => void` – функция, результат которой нас не интересует. Можно вернуть `undefined` или ничего.
- 02 `never: () => never` – функция, которая никогда не вернётся. Тип никогда не имеет значения.
- 03 `void` – supertype `undefined` в некоторых контекстах: переменная `void` может быть `undefined`.
- 04 `never` – bottom type: совместим со всеми типами как `value`, но никакой `value` не имеет тип `never`.
- 05 В `union: T | never = T` (`never` «съедается»). `T | void = T | undefined`.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 `void` – обработчики событий, `void-callbacks` (`forEach`, `addEventListener`).
- 02 `never` – функции, выбрасывающие ошибку: `function fail(): never { throw ... }.`
- 03 `never` – exhaustive check (Q16): default-ветка.
- 04 `never` – фильтр в `conditional types`: тип «отсутствует».

### ПОДВОДНЫЕ КАМНИ

- 01 Тип `T | never` автоматически упрощается до `T` – иногда удивляет, когда ожидаешь полное `union`.
- 02 Функция, которая всегда `throw`, имеет тип `never`. Если явно объявить `void`, теряется информация.
- 03 Передать `() => undefined` туда, где ждут `() => void` – ок. Обратное – НЕ ок.
- 04 В `catch` блоке рекомендуется `(e: unknown)`, не `(e: never)`.

### КОД

```
// void – обычная функция
```

```
function log(msg: string): void {  
  console.log(msg);  
}
```

```
// never – не возвращается
```

```
function fail(msg: string): never {  
    throw new Error(msg);  
}
```

// void в callback – гибкость

```
type Cb = () => void;  
const fn: Cb = () => 42; // ok, возвращаемое значение игнорируется
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Что вернёт T | never и почему?
- 02 Зачем функции тип never?
- 03 Можно ли присвоить undefined переменной типа void?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/functions.html#void>

## ВОПРОС Q15

## Где используется never?

### ОПИСАНИЕ

`never` — тип «нет значения». Используется в нескольких сценариях: возвращаемый тип функций, которые не возвращаются (`throw`, `infinite loop`, `process.exit`); `exhaustive check` в `switch`; результат `conditional/mapped` типов, когда ничего не подходит; пересечение несовместимых типов (`string & number`).

### КАК РАБОТАЕТ

- 01 Возврат-тип `throw`-функций.
- 02 `Exhaustive check`: `const _: never = action` — упадёт, если появится новый вариант `union`.
- 03 `Filter` в `conditional`: `T extends Cond ? Yes : never` — нежеланные ветки выкидываются.
- 04 `Distributive condition`: `union` → `distribute` → каждый вариант проверяется → `never` для не подходящих.
- 05 `Intersection` несовместимых: `'a' & 'b' = never`.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 `function assertNever(x: never): never { throw ... }` — `exhaustive check`.
- 02 `type NonEmpty<T> = T extends [] ? never : T` — фильтр пустых массивов.
- 03 `Exclude<U, T>` через `conditional` + `distributive` — под капотом отбрасывает `never`-ветки.
- 04 Объединение `default`-кейса с `error`-логированием.

### ПОДВОДНЫЕ КАМНИ

- 01 `never` молча «исчезает» в `union` — не путайтесь, когда ожидаете тип, а получаете `T`.
- 02 В разработческом флоу — никогда не присваивайте `never` случайно: TS ничего не примет.
- 03 Если не написать `assertNever`, `exhaustive check` не сработает при появлении нового варианта.
- 04 С `catch-error` — лучше `unknown`, не `never`.

### КОД

// Throw-функция

```
function unreachable(msg = 'unreachable'): never {  
  throw new Error(msg);  
}
```

// Exhaustive switch

```
function reduce(a: Action) {
```

```
switch (a.type) {  
  case 'add':    return a.item;  
  case 'remove': return a.id;  
  default:      const _: never = a; throw new Error('?');  
}  
}
```

// Filter в conditional

```
type NonNullable<T> = T extends null | undefined ? never : T;
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Чем never отличается от void семантически?
- 02 Как используется never в Exclude<T, U>?
- 03 Что делает assertNever и почему он полезен?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#the-never-type>

ДЕНЬ 5

НАГРУЗКА

ТЯЖЁЛЫЙ

2 ВОПРОСА

## ASSERTION FUNCTION

## EXHAUSTIVE CHECK

### ЧТО УЧИМ В ЭТОТ ДЕНЬ

Связка для гарантий: функция-утверждение и проверка-полноты `switch`.

### ВОПРОСЫ ДНЯ

Q11 Что такое `assertion function`?

Q16 Что такое `exhaustive check`?

## Что такое assertion function?

### ОПИСАНИЕ

Assertion function – функция, которая ничего не возвращает, но утверждает свойство аргумента: если аргумент не соответствует – функция выбрасывает. Сигнатура – `asserts x is T` или `asserts x`. После вызова такой функции TS сужает тип аргумента в оставшемся коде. Это похоже на `type guard`, но без `if`-проверки – `assertion` обещает `throw`, если что-то не так.

### КАК РАБОТАЕТ

- 01 `function assertNonNull(x: unknown): asserts x is NonNullable<T> { if (x == null) throw }.`
- 02 Сигнатура `asserts x is T` – после вызова TS считает `x` уточнённым.
- 03 Сигнатура `asserts x` – тождественно `asserts x is true` (TS ставит `truthy`).
- 04 Assertion-функцию обязательно объявлять как `const` или `function`-объявление; стрелочные `require explicit type`.
- 05 Throw обязателен – если функция вернётся без `throw`, `narrowing` сойдёт.
- 06 Можно использовать для аргументов класса/конструктора.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Парсинг `unknown` в конкретный тип после `fetch`.
- 02 Валидация `props` в API: `assertIsUser(data)`.
- 03 Объединение с `runtime`-валидаторами `Zod`: `schema.parse` – это `assertion-like`, но возвращает значение.
- 04 Тестовые помощники: `assertEqual`, `assertExists`.

### ПОДВОДНЫЕ КАМНИ

- 01 Если функция не `throw`, а вернётся – TS всё равно сужает, и баг проскочит в рантайм.
- 02 `asserts x is T` требует вручную поддерживать соответствие проверки и типа.
- 03 В стрелочных функциях нужно явно `typing`: `const assert: (x: unknown) => asserts x is T = ...`
- 04 Не путайте с `as`-кастингом: `as` ничего не проверяет, `assertion`-функция должна реально валидировать.

### КОД

// Базовый `assertion`

```
function assertIsUser(x: unknown): asserts x is User {
  if (!isUser(x)) throw new TypeError('not a user');
}
```



```
function f(x: unknown) {  
  assertIsUser(x);  
  x.name; // ok, narrowed  
}
```

// asserts x – truthy

```
function assertDefined<T>(x: T | undefined): asserts x {  
  if (x === undefined) throw new Error('undef');  
}
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Чем assertion function отличается от type guard?
- 02 Что произойдёт, если assertion-функция не throw?
- 03 Как это связано с Zod / Valibot?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/release-notes/typescript-3-7.html#assertion-functions>

## Что такое exhaustive check?

### ОПИСАНИЕ

Exhaustive check (исчерпывающая проверка) – приём, позволяющий гарантировать, что все варианты union обработаны в switch/if. Достигается через default-ветку, в которой переменная присваивается never. Если появится новый вариант union – TS не сможет присвоить его never, и компиляция упадёт. Это инструмент защиты от пропущенных кейсов при росте кода.

### КАК РАБОТАЕТ

- 01 function assertNever(x: never): never { throw } – вспомогательная функция.
- 02 В default switch вызвать assertNever(action) или присвоить const \_: never = action.
- 03 Если все кейсы обработаны – переменная action в default имеет тип never, всё ок.
- 04 Если добавили новый вариант в union и забыли case – TS не сможет привести action к never, компиляция падает.
- 05 Работает только с discriminated union или явными сужениями.
- 06 Нельзя обойтись только default: throw – TS не проверит exhaustive.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Reducer с discriminated union actions.
- 02 Mapping состояний state-machine.
- 03 Switch по discriminator поля type/kind.
- 04 Бизнес-логика с конечным набором значений (плата, статус, роль).

### ПОДВОДНЫЕ КАМНИ

- 01 Без assertNever – типичная ошибка пропустить новый кейс.
- 02 Вместо never можно использовать absurd-функцию любой библиотеки (ts-essentials и т.п.).
- 03 Если discriminator не literal – narrowing не работает, exhaustive check тоже.
- 04 Switch без break – каскадное падение и неверное narrowing.

### КОД

// Каноническая форма

```
function assertNever(x: never): never {  
  throw new Error('Unreachable: ' + JSON.stringify(x));  
}  
  
function reduce(a: Action) {  
  switch (a.type) {
```

```
    case 'add':    return ...;
    case 'remove': return ...;
    default:      return assertNever(a);
  }
}
```

// Без assertNever – TS не проверит

```
function bad(a: Action) {
  if (a.type === 'add')    return 1;
  if (a.type === 'remove') return 2;
  // забыли 'clear' – TS промолчит
}
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Что такое assertNever и зачем оно?
- 02 Чем exhaustive check лучше default-ветки с throw?
- 03 Когда exhaustive check не работает?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#exhaustiveness-checking>

**IMPOSSIBLE STATES****КОГДА any****OPTIONAL**

## ЧТО УЧИМ В ЭТОТ ДЕНЬ

Дизайн-практики: как не давать невозможным состояниям выразиться.

## ВОПРОСЫ ДНЯ

Q07 Как сделать impossible states unrepresentable?

Q13 Когда допустимо использовать any?

Q17 Что такое optional property?

## ВОПРОС Q07

## Как сделать impossible states unrepresentable?

### ОПИСАНИЕ

Идея – спроектировать тип так, чтобы невозможные комбинации значений были невыразимы на уровне типа. Классический пример – async-state: вместо одного объекта { loading; data; error }, где все три могут существовать одновременно (что бессмысленно), описать его как discriminated union. Тогда компилятор сам не даст написать одновременно loading=true и data.

### КАК РАБОТАЕТ

- 01 Заменить «плоский» объект на discriminated union по тегу.
- 02 Использовать literal types для маркеров состояний.
- 03 Использовать never для исключения нелогичных полей.
- 04 Использовать readonly + const для гарантии иммутабельности.
- 05 Использовать generic-параметры для связывания связанных полей (например, событие с конкретным payload).

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Async-state: loading | success | error.
- 02 Form validation: untouched | dirty | submitting | submitted | error.
- 03 API payload: success | error (Result-тип).
- 04 Permissions: разные роли – разные доступные действия.

### ПОДВОДНЫЕ КАМНИ

- 01 Слишком много discriminated union вариантов делает код многословным.
- 02 Тип становится сложнее – иногда команды отказываются из-за читаемости.
- 03 Без exhaustive check (Q16) – недополучаем выгоду.
- 04 Нельзя исправить уже плохо спроектированный legacy-стейт без рефакторинга.

### КОД

// Плохо – все поля одновременно

```
type State = {
  loading: boolean;
  data?: User;
  error?: Error;
};
// loading=true && data – возможно. Это бессмысленно.
```

// Хорошо – discriminated union

```
type State =
  | { kind: 'idle' }
```

```
| { kind: 'loading' }  
| { kind: 'success'; data: User }  
| { kind: 'error'; error: Error };
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Что выбрать для async-state и почему?
- 02 Какие cost'ы у такого моделирования?
- 03 Связано ли это с exhaustive check?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions>

## ВОПРОС Q13

## Когда допустимо использовать any?

### ОПИСАНИЕ

Эмпирическое правило: any допустим, когда альтернатива хуже — например, в interop со старым JS-кодом без типов, в одиночных полифиллах, в очень динамичных утилитах. Во всём остальном — лучше unknown, generic, или правильный union. any — это маркер «здесь типизация сдалась», и за ней должна стоять явная причина.

### КАК РАБОТАЕТ

- 01 any совместим со всем — отключает проверки.
- 02 Включается через явное as any или поле без типа в noImplicitAny=false.
- 03 Проникает по graph типов: x: any → x.b: any → x.b.c: any.
- 04 В режиме strict есть noImplicitAny — все нетипизированные параметры подсвечиваются.
- 05 Альтернативы: unknown с проверкой, generic <T>, union конкретных типов.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Interop с no-typed JS-зависимостью, для которой нет @types и времени писать declarations.
- 02 Очень динамическая утилита, где любой контракт ложен (reflect-helpers, ad-hoc data-tools).
- 03 Тесты-моки, где быстро удобнее.
- 04 Временный workaround, помеченный TODO.

### ПОДВОДНЫЕ КАМНИ

- 01 any в одной точке распространяется по всей цепочке присваиваний.
- 02 any блокирует автодополнение — продуктивность падает.
- 03 Зависимости с any внутри своих типов «травят» весь код, использующий их.
- 04 as any часто скрывает реальную проблему типизации, которую лучше решить.
- 05 Code-review — основной фильтр от any: всякий новый any должен иметь обоснование.

### КОД

// Объяснённое использование

```
// TODO: переписать под Zod после рефакторинга API
const data = JSON.parse(raw) as any;
```

// Лучшая альтернатива

```
const data: unknown = JSON.parse(raw);
if (isUser(data)) { /* data: User */ }
```

### МОГУТ ДОСПРОСИТЬ

- 01 Что лучше – any или unknown в утилитах?
- 02 Как ловить проникновение any в проекте?
- 03 Какие линт-правила помогают?

---

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#the-any-type>



## ВОПРОС Q17

## Что такое optional property?

### ОПИСАНИЕ

Optional property – поле объекта с `?`, которое может отсутствовать. Если поле опциональное, его тип становится `T | undefined` (то есть либо настоящее значение, либо `undefined`, либо вообще отсутствие ключа). В `strict`-режиме опциональные поля и поля типа `undefined` – разное поведение (см. `exactOptionalPropertyTypes`).

### КАК РАБОТАЕТ

- 01 `{ x?: number }` – поле `x` может быть `omitted` или `undefined`.
- 02 Эквивалент: `{ x: number | undefined }` – поле обязано присутствовать, но может быть `undefined`. Различие контролируется флагом `exactOptionalPropertyTypes`.
- 03 Доступ: `obj.x` – тип `number | undefined` в любом случае.
- 04 Опциональные поля совместимы с `partial`-апдейтами.
- 05 `Partial<T>` – превращает все поля в опциональные.
- 06 `Required<T>` – снимает все опциональности.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Поля API-ответа, которые могут быть пропущены.
- 02 Параметры функций.
- 03 Конфигурация с дефолтами.
- 04 `Form-state`, где не все поля обязательно заполнены.

### ПОДВОДНЫЕ КАМНИ

- 01 В нестрогом режиме можно присваивать `null` опциональному полю – `strict` ставит запрет.
- 02 `exactOptionalPropertyTypes` (TS 4.4+) различает «нет ключа» и «ключ = `undefined`» – иногда меняет API.
- 03 Делать всё опциональным = потеря смысла модели. Лучше явно разделять обязательные/опциональные.
- 04 JSON-сериализация теряет `undefined` – поле может пропасть из JSON, поэтому семантика «нет поля» важна.

### КОД

// Базовое

```
type User = { id: string; name: string; email?: string };
const u: User = { id: '1', name: 'A' }; // ok
u.email; // string | undefined
```

// Partial / Required

```
type Patch = Partial<User>; // все поля optional
```

```
type Complete = Required<User>; // все поля обязательны
```

```
// exactOptionalPropertyTypes
```

```
// при флаге true:
```

```
const u: User = { id: '1', name: 'A', email: undefined };
```

```
// ошибка – undefined не разрешён, только omit
```

---

МОГУТ ДОСПРОСИТЬ

01 Чем отличается `?:` и `| undefined`?

02 Что делает `exactOptionalPropertyTypes`?

03 Чем `Partial<T>` опасен в API-моделях?

---

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/objects.html#optional-properties>

ДЕНЬ 7

НАГРУЗКА

ТЯЖЁЛЫЙ

3 ВОПРОСА

**INDEXED ACCESS****MAPPED****CONDITIONAL****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Три механизма манипуляции типами — заход в advanced.

**ВОПРОСЫ ДНЯ**

Q18 Что такое indexed access type?

Q19 Что такое mapped type?

Q20 Что такое conditional type?

## Что такое indexed access type?

### ОПИСАНИЕ

Indexed access type – способ получить тип поля по ключу из другого типа: `T[K]`. Похоже на обращение к свойству в значении, но на уровне типов. Полезно для повторного использования вложенных типов без ручного дублирования. Работает с обычными объектами, массивами и tuple.

### КАК РАБОТАЕТ

- 01 `T[K]` – тип поля `K` объекта `T`.
- 02 `T[keyof T]` – union всех значений `T` (всех типов полей).
- 03 `Array<T>[number]` – тип элемента массива.
- 04 `Tuple[0]`, `Tuple[1]` – типы конкретных позиций кортежа.
- 05 Можно цеплять: `User['address']['city']`.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Чтобы взять тип одного поля existing типа: `type Name = User['name']`.
- 02 Тип элемента массива: `type Item = Items[number]`.
- 03 Тип ключа в объекте через `keyof` + `indexed access` – ядро многих utility-типов.
- 04 Связывание типа функции и её параметра: `Parameters<F>[0]`.

### ПОДВОДНЫЕ КАМНИ

- 01 `T[K]` требует, чтобы `K` был валидным ключом `T` – иначе ошибка.
- 02 Если `T` – union и `K` есть только в части – TS будет ругаться.
- 03 Для массива нужен `T[number]`, не `T[0]`.
- 04 `Indexed access` тяжелее по компилятору, чем явное имя.

### КОД

// Базовое

```
type User = { id: string; address: { city: string } };
type ID   = User['id'];           // string
type City = User['address']['city']; // string
```

// Тип элемента массива

```
const items = ['a', 'b'] as const;
type Item = (typeof items)[number]; // 'a' | 'b'
```

// Tuple

```
type Pair = [number, string];
type Fst  = Pair[0]; // number
```

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое keyof и как он сочетается с indexed access?
- 02 Как взять тип элемента массива?
- 03 Зачем T[number] вместо T[0]?

---

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/indexed-access-types.html>

## Что такое mapped type?

### ОПИСАНИЕ

Mapped type – способ создать новый тип, пройдясь по ключам существующего и трансформировав каждое поле. Базовый синтаксис: `{ [K in keyof T]: F<T[K]> }`. На основе mapped types построены `Partial`, `Required`, `Readonly`, `Pick`, `Record`, и многие другие utility-типы. С TS 4.1+ можно также переименовывать ключи через `as ClauseKey`.

### КАК РАБОТАЕТ

- 01 `{ [K in Keys]: ValueType }` – итерация по объединению ключей.
- 02 Часто используют `keyof T` для итерации по полям типа.
- 03 `Modifiers`: `?:` (optional), `readonly`, и их минус-версии (`-?:`, `-readonly`) для удаления.
- 04 `Key remapping`: `{ [K in keyof T as NewKey]: ... }` – переименование ключей через `template literal types`.
- 05 `Distribution` через `union` (см. `Q-distributive` в `advanced`).

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Partial<T>`, `Required<T>`, `Readonly<T>`, `Pick<T, K>`, `Record<K, V>` – все построены на mapped types.
- 02 `Custom-helpers`: `type Nullable<T> = { [K in keyof T]: T[K] | null }.`
- 03 `API-shape transformation`: rename ключей при сериализации.
- 04 `Generic data-loaders`: загрузить разные сущности по ключу.

### ПОДВОДНЫЕ КАМНИ

- 01 Не сохраняет порядок полей в типе (как и обычный объект).
- 02 `Modifiers` `?:` и `readonly` применяются ко всем полям – иногда нужно явно отключать через `-`.
- 03 `Key remapping` через `template literal` требует TS 4.1+.
- 04 Большие mapped types по большим объектам нагружают компилятор.

### КОД

// Базовое – Partial

```
type MyPartial<T> = { [K in keyof T]?: T[K] };
type X = MyPartial<{ a: string; b: number }>;
// { a?: string; b?: number }
```

// Modifiers

```
type MyReadonly<T> = { readonly [K in keyof T]: T[K] };
type Mutable<T>    = { -readonly [K in keyof T]: T[K] };
```

// Переименование (TS 4.1+)

```
type Getters<T> = {  
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K]  
};  
type X = Getters<{ name: string }>; // { getName: () => string }
```

---

#### МОГУТ ДОСПРОСИТЬ

- 01 Чем mapped type отличается от Pick?
- 02 Что такое key remapping?
- 03 Как сделать deep partial?

---

#### ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/mapped-types.html>

## Что такое conditional type?

### ОПИСАНИЕ

Conditional type – тип, который выбирается на основе проверки условия: `T extends U ? X : Y`. Похоже на тернарный оператор, но на уровне типов. Это базовый инструмент advanced-типизации: на нём построены `ReturnType`, `Parameters`, `Awaited`, `Exclude`, `Extract`, и т.д. С `infer` (см. Q-infer) – позволяет извлекать вложенные типы.

### КАК РАБОТАЕТ

- 01 `T extends U ? X : Y` – если `T` присваиваемо `U`, тип `X`, иначе `Y`.
- 02 `Distributive` над `union`: если `T` – `union`, `conditional` применяется к каждому варианту отдельно (см. Q-distributive в advanced).
- 03 `infer P` – захват подтипа в позиции `extends`-условия. Пример: `T extends (...args: any) => infer R ? R : never` – `ReturnType`.
- 04 Можно вкладывать: `T extends string ? T extends 'a' ? 1 : 2 : 3`.
- 05 Часто комбинируется с `mapped types` для глубокого преобразования.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Извлечение возвращаемого типа: `ReturnType<F>`.
- 02 Извлечение параметров: `Parameters<F>`.
- 03 `Awaited<T>` – рекурсивная распаковка `Promise`.
- 04 Custom: `NonNullable`, `Exclude`, `Extract`.
- 05 Утилиты для domain-типов: `SanitizeApiResponse<T>`.

### ПОДВОДНЫЕ КАМНИ

- 01 `Distributive` поведение часто сюрприз: `T extends U ? X : Y` над `union T = A | B` → `(A extends U ? X : Y) | (B extends U ? X : Y)`.
- 02 Чтобы отключить `distribution` – обернуть в `[T] extends [U] ? X : Y`.
- 03 `infer` работает только в позиции `extends`.
- 04 Слишком сложные `conditional` замедляют компилятор и ухудшают читаемость ошибок.

### КОД

// Базовое

```
type IsString<T> = T extends string ? 'yes' : 'no';
type A = IsString<'x'>; // 'yes'
type B = IsString<42>; // 'no'
```

// С `infer` – `ReturnType`

```
type ReturnType<F> = F extends (...args: any) => infer R
  ? R
  : never;
```



// Distributive vs non-distributive

```
type Distrib<T>    = T extends string ? T : never;
type NonDistrib<T> = [T] extends [string] ? T : never;
// Distrib<'a' | 1> = 'a'
// NonDistrib<'a' | 1> = never
```

---

МОГУТ ДОСПРОСИТЬ

- 01 Что такое distributive conditional?
- 02 Зачем нужен infer?
- 03 Как написать Awaited<T>?

---

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/conditional-types.html>

# СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ    БЕЗ ПОДГЛЯДЫВАНИЙ    ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Сравнить `interface` и `type`, описать `declaration merging` (Q1, Q2)
- ☐ Описать `union`, `intersection`, `discriminated union` с примерами (Q3, Q4, Q6)
- ☐ Объяснить `literal types` и `as const` (Q5)
- ☐ Расписать `narrowing` через `typeof` / `in` / `instanceof` (Q8, Q9)
- ☐ Написать `user-defined type guard` и `assertion function` (Q10, Q11)
- ☐ Сравнить `unknown` и `any`, объяснить когда `any` допустим (Q12, Q13)
- ☐ Развести `never` и `void`, привести 3 случая `never` (Q14, Q15)
- ☐ Описать `exhaustive check` через `assertNever` (Q16)
- ☐ Объяснить, как делать `impossible states unrepresentable` (Q7, Q17)
- ☐ Расписать `indexed access`, `mapped`, `conditional types` (Q18, Q19, Q20)